BREAK THROUGH TECH AI STUDIO · BANKING 1B

Online Payment Fraud Detection

Progress update: data, features, baselines and first neural networks

Team

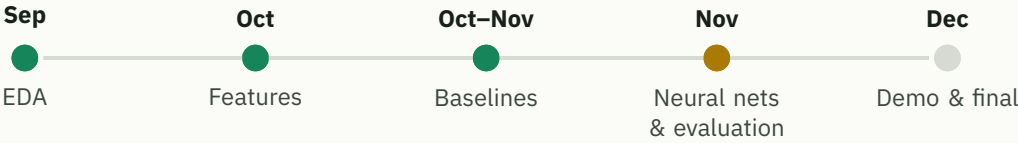
Kyaw Thi (Eric) Ha · Salamot Fakoya · Jean Cho · Jennifer Perez-Juarez · Salvador Frias · Susie Marrero · Bianca Blevins

Challenge Advisor Debasmita Das **AI Studio Coach** Harshini Donepudi

November 2026 · IEEE-CIS Fraud Detection dataset · all results on the validation period; holdout untouched

WHERE WE ARE

Three stages done, evaluation under way



Done

Joined and explored the data, built two feature tables, trained logistic regression, LightGBM and XGBoost.

This month

Small neural networks, a same-machine comparison with LightGBM, blending and segment checks.

Today we need

Your view on the detection target and review capacity, and on what to tune before the holdout test.

DATA & SETUP

Fraud is rare, so we split by time and keep a final test untouched

590,540
labeled transactions (IEEE-CIS, 434 joined columns)

3.5%
are fraud in the training period

0.031
average precision of random guessing on validation



earlier transactions → later transactions

59,054 validation · 59,054 holdout

Every model learns only from older transactions and is judged on later ones, like real deployment. The holdout has **not been scored yet**: it is saved for the single final model.

EXPLORATORY ANALYSIS · WHOLE TEAM

What the data told us

Fraud is rare	3.51% of training transactions. Accuracy would hide poor detection, so we use precision–recall metrics.
Product mix matters	Product C has an 11.3% fraud rate; high-volume product W holds about 43% of all fraud.
Missingness is a signal	414 of 434 columns have gaps, in repeating blocks whose fraud rates differ.
Address gaps hide risk	Transactions with no billing address are 11.4% of rows but 36.9% of fraud, so we built a second account key for them.

Source: combined EDA notebook and EDA summary notes.

Two model-ready tables built only from past information

- **Frequency and out-of-time target encoding** for categories with many values
- **Account history and velocity**: counts and amounts over 1 h, 24 h, 7 and 30 days, using strictly earlier transactions
- **Missingness** flags per block and missingness-profile clusters
- **Email, device and amount** features, `id_14` timezone as a category
- **Reduced V-column set** (drop near-duplicates, $|r| > 0.95$)

Tree table **537 columns**

Categories and missing values kept as they are. Used by LightGBM and XGBoost.

Neural-net table **515 columns**

Log-scaled, clipped, filled, standardized; one-hot for small categories. Used by logistic regression and neural nets.

HOW WE EVALUATE

One ranking score, plus the numbers a fraud team would feel

Metric	Plain meaning	Better is
Average precision (AP)	Main score. Summarizes precision across all catch rates; random ≈ 0.031	higher
Recall at 1% FPR	Share of fraud caught if we may flag only 1% of legitimate customers	higher
TFPR at 50% detection	Legitimate alerts per fraud caught when we catch half the fraud	lower
Top-5% capture	Share of fraud found if analysts review only the riskiest 5%	higher

All models use the team's shared `evaluate()` function on the same 59,054 validation transactions.

Boosted trees beat the linear baseline by a wide margin

Model (validation, untuned)	AP	ROC-AUC	Recall @1% FPR	Top-5% capture
Logistic regression (numeric table)	0.428	0.867	37.6%	50.8%
XGBoost (integer-coded categories, 3 seeds)	0.613	–	53.1%	–
LightGBM (`all` features, tree table)	**0.632**	**0.931**	**56.4%**	**68.4%**

Feature checks. Removing engineered families (target encoding, behavior) changed AP in different directions across seeds: no consistent gain or loss yet, so nothing is dropped.

Leakage check. Training on shuffled labels gives AP 0.033, the same as random (0.031). No sign of leakage in the label-free features.

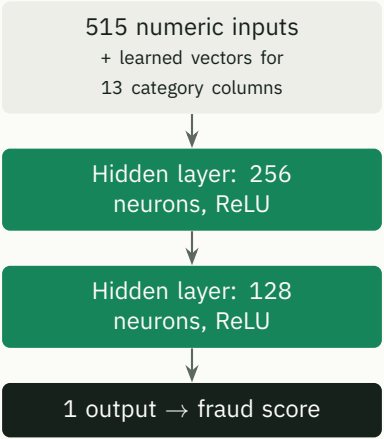
Source: baseline modeling notebook, Sections 6d3, 8 and 13. XGBoost recall is seed 0.

NOVEMBER · SMALL NEURAL NETWORKS · ERIC

Logistic regression adds up features in a straight line. Can a small neural network learn the combinations the trees find?

Same 515-column table, same rows, same metrics. One change at a time.

What we trained



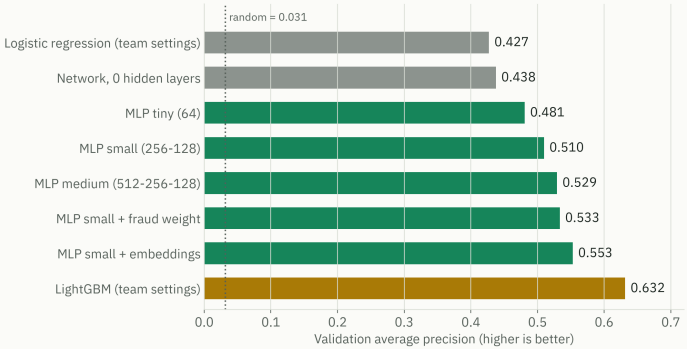
Training. Mini-batches of 2,048, Adam (lr 0.001), batch norm, dropout, early stopping on validation AP (patience 4).

Experiments.

- 0 hidden layers (control)
- Size: 64 · 256–128 · 512–256–128
- Extra weight on fraud rows (5.2×)
- **Embeddings** for `card1`, `addr1`, email domains, device and browser

About 1 minute per model on a laptop CPU.

Hidden layers and embeddings each add real lift; trees still lead



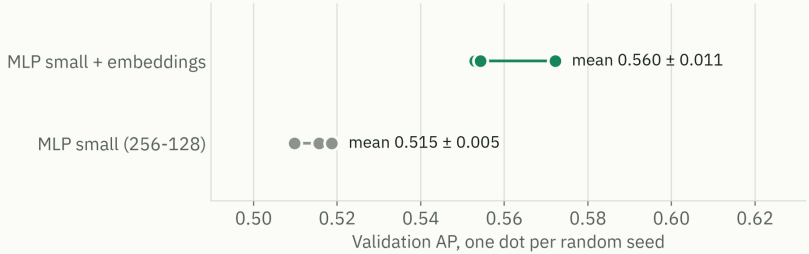
Validation AP, seed 0, all models trained on the same machine and rows. Gray = linear, emerald = neural nets, amber = LightGBM (team settings).

What each change was worth

Change	AP	What it means
Training loop alone (0 hidden layers)	0.438	Matches logistic regression (0.427): the gains below come from hidden layers
Add one hidden layer (64)	0.481	Non-linear combinations help immediately
Bigger: 256–128 / 512–256–128	0.515 / 0.529	Diminishing returns from size
Weight fraud rows 5.2×	0.533	+0.02 on one seed; promising, not yet confirmed
Embeddings for card, address, email, device	**0.560**	**+0.045**, the largest single gain (3-seed mean)

Why embeddings help: `card1` and `addr1` are ID codes; as plain numbers, ID 9500 looks “bigger” than 7919. An embedding gives each common ID its own learned vector.

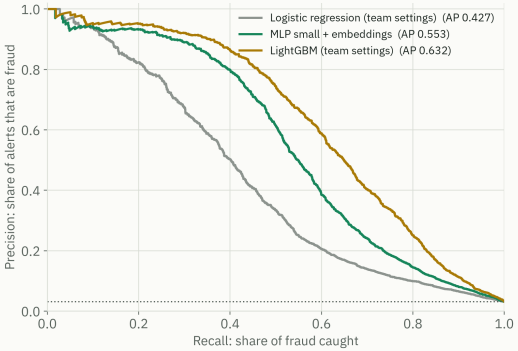
The embedding gain is far larger than seed-to-seed noise



Each dot is one training run with a different random start. The two ranges do not overlap: the worst embedding run (0.553) beats the best plain run (0.519).

COMPARISON

Best neural net against LightGBM, same validation rows



	LogReg	Net	LGBM
AP	0.427	0.553	**0.632**
ROC-AUC	0.867	0.903	**0.931**
Recall @1% FPR	37.6%	49.9%	**56.4%**
TFPR @50% det.	1.99	0.63	**0.34**
TFPR @80% det.	9.08	5.88	**2.98**
Top-5% capture	50.8%	60.6%	**68.4%**

The net closes about **60%** of the gap from logistic regression to LightGBM. At 90% detection: 14.3 / 11.3 / 7.8 legitimate alerts per fraud. **Only LightGBM stays under 10:1.**

COMPARISON · BLENDING

Does the network add anything on top of LightGBM?

Rank-average blends

Net weight	AP	vs LightGBM
0% (LightGBM alone)	0.632	–
10%	0.628	−0.004
20%	0.623	−0.008
30%	0.619	−0.013
50%	0.608	−0.023

95% time-block bootstrap intervals all stay below 0 (10%: −0.007 to −0.0001).

Who catches which fraud (top 5% reviewed)



- Both: 1,061 (57%)
- Only LightGBM: 205 (11%)
- Only the net: 60 (3%)
- Missed by both: 525 (28%)

The net mostly finds frauds LightGBM already finds, so **blending does not help**. The 28% both miss points to missing information, not model choice.

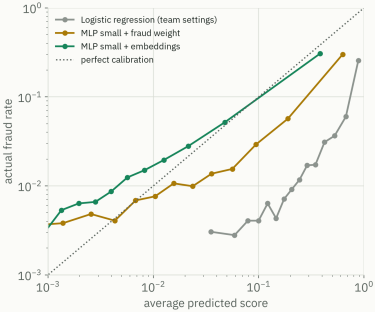
COMPARISON · WHERE IT WORKS

Segments and calibration

AP within each segment (compare across a row)

Segment	Fraud	LogReg	Net	LGBM
Product C	11.8%	0.731	0.756	0.804
Product R	4.7%	0.724	0.814	0.858
Product W	1.9%	0.116	0.356	0.461
Identity present	8.9%	0.684	0.734	0.787
Identity missing	1.9%	0.118	0.358	0.462
Address missing	12.1%	0.738	0.758	0.806

The net trails LightGBM in every segment; the gap is largest (≈0.10) on product W and no-identity transactions. On product W it triples logistic regression's AP.



Unweighted net: average score 0.033 vs 0.031 actual fraud rate, so its **scores read as real probabilities**. Weighting inflates them.

TAKEAWAYS

What we can say today

+0.13 AP

Small nets beat the linear baseline

0.427 → 0.560 on the same inputs, from hidden layers plus embeddings. Repeatable over 3 seeds.

−0.07 AP

LightGBM is still the strongest

Ahead on every metric and segment; blending the net in makes it slightly worse.

28%

of fraud is missed by both

At a 5% review budget. The next gains are more likely from information than from a new algorithm.

Recommendation: keep LightGBM as the main model, keep the embedding network as the documented deep-learning baseline, and score the holdout once the model is frozen.

Caveats: validation is used for early stopping and model choice (slightly optimistic); most comparisons are one seed; no model is tuned yet.

NEXT STEPS

Plan to December, and questions for you

Next steps

- Combine what helped (embeddings + fraud weighting), 3 seeds
- Embeddings for the team's account keys
- Study the 525 frauds both models miss, mostly product W
- Tune LightGBM / XGBoost; confirm threshold with the TFPR table
- Freeze model and features, then score the holdout once
- December: demo, SHAP explanations, final README

Questions for our advisor

1. Which detection target and review capacity should the final threshold use? (Validation stays under 10 legitimate alerts per fraud up to 90% detection.)

2. Is a small blend acceptable, or should the final model be a single model for explainability?

3. Should we spend remaining time on tuning trees or on sequence models over account histories?